Clearance-Constrained PCB Global Placement with Heterogeneous Components

Yan-Jen Chen[1], Wei-Kai Huang[1], Chung-Ting Tsai[2], Chiao-Yu Ou[1], and Yao-Wen Chang[1,2]
[1]Graduate Institute of Electronics Engineering, National Taiwan University, Taipei 106319, Taiwan
[2]Department of Electrical Engineering, National Taiwan University, Taipei 106319, Taiwan
yjchen@eda.ee.ntu.edu.tw; wkhuang@eda.ee.ntu.edu.tw; b09901020@ntu.edu.tw; cyou@eda.ee.ntu.edu.tw; ywchang@ntu.edu.tw

2025 62nd ACM/IEEE Design Automation Conference (DAC) | 979-8-3315-0304-8/25/$31.00 ©2025 IEEE | DOI: 10.1109/DAC63849.2025.11133056

***Abstract*—The complexity of design rules and intense time-to-market demands have made auto-placement tools essential for advanced printed circuit board (PCB) designs. This paper presents a novel PCB placement framework to handle pad-to-pad clearance constraints and heterogeneous components to address these challenges. Unlike existing academic placers, our framework focuses on the following key features: a wire-area model to account for various routing resource needs between power and signal nets, a pad-to-pad clearance model to minimize spacing violations, and a two-sided, pad-type-aware density model to reduce component and pad overlap. We further develop a quadratic programming-based legalizer to resolve constraint violations among components of varying shapes. Experimental results show the effectiveness and efficiency of our framework, surpassing two state-of-the-art academic placers in post-routing quality on both academic and industrial benchmarks.**

I. Introduction

The rapidly evolving demands in semiconductor devices have driven leading electronics companies to seek increasingly shorter printed circuit board (PCB) design cycles to maintain competitive time-to-market requirements. Furthermore, the rise of high-performance computing servers, artificial intelligence (AI) accelerators, and the Internet of Things (IoT) has made these design cycles more challenging. As a result, modern PCB designs incorporate more components and requires more complex design rules, inevitably lengthening the design cycles.

For the PCB design flow, physical design is among the most time-intensive stages. Experienced engineers need to find a desired layout by iterating placement and routing processes, requiring extensive expertise and even time-consuming human efforts. Existing commercial and open-source electronic design automation (EDA) tools, including Allegro [1], Altium [2], and KiCad [3], primarily focus on supporting manual placement, auto-routing, and design-rule checking. There is an industry-wide push to develop advanced PCB placement algorithms that could replace manual placement and expedite this critical design stage to shorten the design cycle [4], [5]. For example, Renesas' [6] recent $5.9 billion acquisition of Altium in February 2024 highlights the growing importance of PCB design automation in accelerating innovation.

The heterogeneous nature of PCB components and the complexity of associated design rules pose substantial challenges to placement algorithms. Figure 1 illustrates an example of heterogeneous components, highlighting unique features: components may have arbitrary shapes, be mounted on either side of the board, and incorporate both surface-mount-device (SMD) pads and through-hole (TH) pads, where only the latter penetrate through the board. Figure 2 shows the complexity of design rules derived from electrical requirements, such as minimizing crosstalk and ensuring insulation distances, which translate to clearance constraints (i.e., minimum spacing requirements between objects). Besides these features, PCBs are routed in a 135-degree style. In contrast, traditional placement algorithms often assume all components are rectangular, requiring only basic non-overlap constraints incompatible with modern PCB designs, and are routed in a 90-degree style. Given these unique challenges, PCB placement is widely regarded as a complex problem that necessitates advanced algorithms to achieve effective and efficient layout solutions.

A. Previous Work

Existing academic approaches to PCB placement can be broadly categorized into three methods: genetic algorithms, analytical placement, and reference placement. Genetic algorithm methods [7], [8] model component coordinates and orientations as chromosomes, optimizing a weighted objective function composed of heuristic metrics, such as thermal performance, timing, and layout tidiness. These algorithms adjust component locations and orientations through swapping or shifts. However, their reliance on random optimization techniques limits scalability, making them less suitable for large, complex designs.

This work was partially supported by AnaGlobe, ASUS, Delta Electronics, Google, Maxeda Technology, TSMC, NSTC of Taiwan under Grant NSTC 110-2221-E-002-177-MY3, NSTC 113-2218-E-002-041,NSTC 113-2223-E-002-007, and NSTC 113-2640-E-002-001.

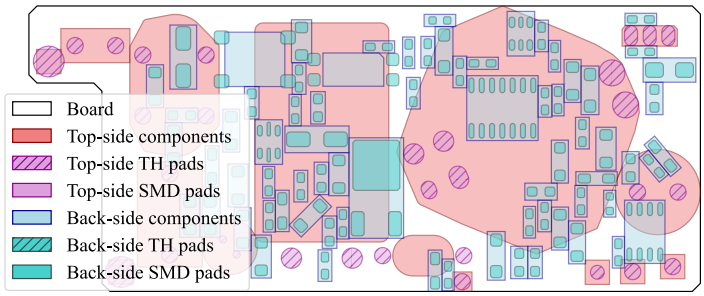
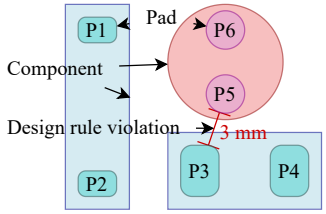


Fig. 1. An example of a modern PCB design with heterogeneous components. The board, components, and pads can have arbitrary shapes, with components rotatable at any angle and placeable on either the top or back side of the board. Through-hole (TH) pads penetrate the board, occupying placement areas on both sides, while surface-mount device (SMD) pads occupy placement areas on a single layer. This dense and complex architecture presents significant challenges for PCB placement automation.

	P1	P2	P3	P4	P5	P6
P1						
P2	0					
P3	1	1				
P4	1	1	0			
P5	5	5	5	5		
P6	5	5	5	5	0	

(a) Pad-to-pad clearance (unit: mm)



(b) Placement with one design rule violation

Fig. 2. An illustration of pad-to-pad clearance constraints in PCB design. (a) The pad-to-pad clearance represents the minimum spacing required between two pads. (b) Design rule violations can arise between arbitrary shapes, making them challenging to detect and resolve.

Analytical placement methods adopt strategies from integrated circuit (IC) placement, applying gradient descent to spread components and minimize objectives like total wirelength, component density penalties, and other additional costs. Cheng *et al.* [9] introduced a net separation regularizer to enhance net separation and improve routability for PCB designs. Huang *et al.* [10] developed a density calculation algorithm using line-drawing and polygon-clipping techniques to estimate density contributions from rotated rectangular components. Tsou *et al.* [11] proposed a force-directed PCB placement framework that optimizes different design intents (e.g., routability, power flow, and clearance between pads) in different optimization stages. Additionally, Chen *et al.* [12] proposed a 2.5D density model to simultaneously place different-layer components. In particular, Tsou *et al.*'s and Chen *et al.*'s approaches accommodate two-sided component placement, whereas most other methods are limited to one-sided placement. Despite these advancements, analytical approaches often model components with simplified rectangular bounding boxes, which can lead to wasted space and impaired spatial efficiency in heterogeneous component layouts. Moreover, while Tsou *et al.*'s method effectively reduces clearance constraint violations, it may introduce large displacements due to the use of a greedy search for legalizing placements.

The reference placement approach, by contrast, attempts to place components in new designs based on established patterns stored in a database. For instance, Su *et al.* [13] developed a subgraph-matching algorithm to enhance the efficiency and quality of pattern matching in PCB layouts. However, this approach requires an extensive pattern database, which is beyond the scope of this paper.

In summary, while these academic placement approaches offer valuable insights, they lack full compatibility with the demands of modern PCB de-

signs. Genetic algorithms struggle with scalability; analytical placements often simplify component shapes in ways that reduce spatial efficiency, and reference placement methods require a pre-existing pattern database, limiting their direct applicability to new designs without extensive data preparation.

B. Our Contributions

This paper adopts an analytical placement approach and presents a new PCB placement algorithm to address the limitations of existing academic placers, particularly with regard to pad-to-pad clearance constraints and heterogeneous components. The main contributions of our algorithm are as follows:

- To the best of our knowledge, this paper is the first PCB global placement work considering pad-to-pad clearance constraints and heterogeneous components.
- A wire-area model is introduced to calculate the total routing resource consumption for power and signal nets, establishing better placement optimization.
- A pad-to-pad clearance model is proposed to minimize spacing violations between pads, enhancing placement quality and routability.
- A two-sided, pad-type-aware density model is developed to reduce overlaps between heterogeneous components and pads. This approach supports co-optimization across both PCB sides and is particularly suited for dense PCB designs.
- A quadratic programming legalizer is proposed to resolve remaining overlaps between heterogeneous components and pads. This approach addresses runtime bottlenecks, significantly enhancing the scalability of the PCB placement algorithm.
- Experimental results on both academic and industrial PCB designs demonstrate the effectiveness and efficiency of our algorithm. The proposed placement algorithm achieves the best post-routing result compared to two state-of-the-art academic PCB placers. Specifically, there is an average 30% post-routing wirelength reduction in the academic designs, an average 74% clearance violations reduction, and an average 32% routability improvement in the industrial designs.

The remainder of this paper is organized as follows. Section II states our problem formulation and introduces our analytical placement framework. Section III gives our overall design flow. Sections IV and V detail our algorithms. Section VI reports the experimental results. Section VII gives the conclusion.

II. Preliminaries

This section introduces the PCB placement problem and our analytical placement framework. Table I lists the notations used in this paper.

A. Problem Formulation

The PCB placement problem can be formulated as a hypergraph $G = (V, E)$ placement problem, where the set of vertices V represents components and the set of hyperedges E represents nets. Let x_i and y_i denote the x and y coordinates of component v_i, respectively. Components are pre-assigned to either the top side or back side of the board, may have arbitrary shapes, and may be pre-placed at a fixed location. Each component can include up to two types of pads: surface-mount device (SMD) pads and through-hole (TH) pads. All pads are assumed to have convex shapes. In particular, TH pads penetrate the board, occupying space on both sides. Let w_e denote the wire-width of a net e. We intend to determine the desired position of each component such that the target cost (i.e., total wire area) is minimized and the following constraints are satisfied:

- **Nonoverlapping constraints.** Components on the same layer must not overlap. Additionally, components must not overlap with TH pads located on the opposite side of the board.
- **Pad-to-pad clearance constraints.** The minimum distance between any pair of two pads, p_m and p_n, must be at least $c_{m,n}$ units, where $c_{m,n}$ is a user-defined value.

B. Our Analytical Placement Framework

The PCB placement problem is solved in two main stages: (1) global placement, which aims to efficiently find a solution with minimal constraint violations, and (2) legalization, which resolves any remaining violations. Global placement is crucial, as it largely determines the quality of the final solution. In our approach, the global placement problem is formulated as a constrained optimization problem defined as follows:

$$\begin{aligned} \min_{\mathbf{x},\mathbf{y}} \quad & W(\mathbf{x},\mathbf{y}), \\ \text{s.t.} \quad & C_{p2p}(\mathbf{x},\mathbf{y}) = 0, \\ & D(\mathbf{x},\mathbf{y}) = 0, \end{aligned} \tag{1}$$

TABLE I
Notations used in this paper.

V	Set of components.
$V_t(V_b)$	Set of top-side (back-side) components.
E	Set of nets.
$P_t^S(P_b^S)$	Set of top-side (back-side) SMD pads.
P^T	Set of TH pads.
O	Set of obstacles.
$(\mathbf{x},\mathbf{y})$	x and y coordinates of all components.
(x_i, y_i)	x and y coordinates of component v_i.
$W(\mathbf{x},\mathbf{y})$	Total wirelength.
$C_{p2p}(\mathbf{x},\mathbf{y})$	Total pad-to-pad clearance violations.
$D(\mathbf{x},\mathbf{y})$	Density penalty.
$p(p_m, p_n)$	Pad-to-pad clearance penalty between p_m and p_n.
$d(p_m, p_n)$	Minimum distance between pads p_m and p_n.
$c_{m,n}$	Clearance between pads p_m and p_n.
w_e	Wire width of net e.
M_s	Vertex matrix of a shape s.

where $W(\mathbf{x},\mathbf{y})$, $C_{p2p}(\mathbf{x},\mathbf{y})$, and $D(\mathbf{x},\mathbf{y})$ represent the total wirelength, total pad-to-pad clearance violations, and the density penalty (indicating the degree of component overlaps), respectively. To convert Equation 1 into an unconstrained optimization problem, we apply Lagrange relaxation as follows:

$$\min_{\mathbf{x},\mathbf{y}} \quad W(\mathbf{x},\mathbf{y}) + \lambda_1 C_{p2p}(\mathbf{x},\mathbf{y}) + \lambda_2 D(\mathbf{x},\mathbf{y}), \tag{2}$$

where λ_1 and λ_2 are the Lagrange multipliers. Equation 2 is then solved using gradient descent, with iterative increases in λ_1 and λ_2.

To better capture the 135-degree routing style typically seen in PCB design, the total wirelength $W(\mathbf{x},\mathbf{y})$ is modeled using the X half-perimeter wirelength (XHPWL) [14] instead of the traditional half-perimeter wirelength (HPWL) used in IC placement:

$$\begin{aligned} W(\mathbf{x},\mathbf{y}) &= \sum_{e\in E} l_e(\mathbf{x},\mathbf{y}), \\ l_e(\mathbf{x},\mathbf{y}) &= \left(\sqrt{2}-1\right)\left(\max_{v_i,v_j\in e}|x_i - x_j| + \max_{v_i,v_j\in e}|y_i - y_j|\right) - \\ &\quad \left(\frac{\sqrt{2}}{2}-1\right)\left(\max_{v_i,v_j\in e}|(x_i+y_i) - (x_j+y_j)| + \right. \\ &\quad \left. \max_{v_i,v_j\in e}|(x_i-y_i) - (x_j-y_j)|\right), \end{aligned} \tag{3}$$

where $e \in E$ represents a net and $l_e(\mathbf{x},\mathbf{y})$ denotes the XHPWL of net e. Since XHPWL is non-differentiable, we use a weighted-average model [15] for a smooth approximation.

The total pad-to-pad clearance violation $C_{p2p}(\mathbf{x},\mathbf{y})$ is defined as the sum of insufficient distances between all pairs of pads on each side of the board:

$$\begin{aligned} C_{p2p}(\mathbf{x},\mathbf{y}) = &\sum_{p_m,p_n\in P_t^S\cup P^T} p(p_m,p_n) + \\ &\sum_{p_m,p_n\in P_b^S\cup P^T} p(p_m,p_n), \\ p(p_m,p_n) = &\max\left(c_{m,n} - d(p_m,p_n), 0\right), \end{aligned} \tag{4}$$

where P_t^S (P_b^S), P^T, and $p(\cdot)$ are the sets of top-side (back-side) SMD pads, TH pads, and pad-to-pad clearance penalty between two pads, respectively, and $d(\cdot)$ represents the minimum distance between pads. To calculate $d(\cdot)$ for pads with arbitrary convex shapes, we employ the Gilbert-Johnson-Keerthi (GJK) distance algorithm [16] and the expanding polytope algorithm (EPA) [17].

The density penalty $D(\mathbf{x},\mathbf{y}) = D_t(\mathbf{x},\mathbf{y}) + D_b(\mathbf{x},\mathbf{y})$ comprises the top-side density penalty $D_t(\mathbf{x},\mathbf{y})$ and back-side density penalty $D_b(\mathbf{x},\mathbf{y})$, similar to the approach used by Chen *et al.* [12]. $D_t(\mathbf{x},\mathbf{y})$ is defined as the eDensity model [18] to minimize component overlaps:

$$D_t(\mathbf{x},\mathbf{y}) = \sum_{v_i\in V_t} q_i \phi(x_i, y_i), \tag{5}$$

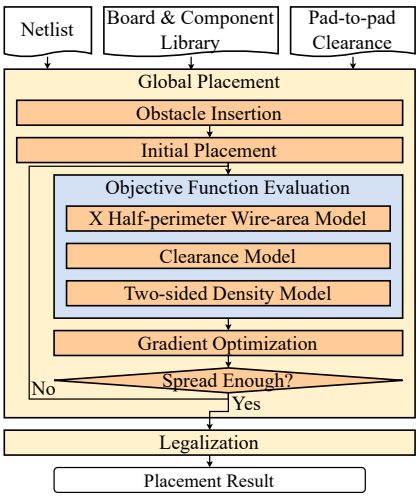


Fig. 3. The overall flow of our algorithm.

where V_t is the set of top-side components, q_i is the area of component v_i, and $\phi(x_i, y_i)$ is its electric potential, computed using spectral methods and the fast Fourier transform. The placement region is divided into $m_x \times m_y$ bins, each associated with a frequency index $(\omega_i, \omega_j) = (\frac{\pi i}{m_x}, \frac{\pi j}{m_y})$. The density map $\rho(x, y)$ can be calculated using these bins, representing the total area of components within each bin divided by the bin area. The electric potential $\phi(x, y)$ is then calculated as follows:

$$\begin{aligned} a_{i,j} &= \frac{1}{m_x m_y} \sum_{x,y} \rho(x,y) \cos(\omega_i x) \cos(\omega_i y), \\ \phi(x,y) &= \sum_{i,j} \frac{a_{i,j}}{\omega_i^2 + \omega_j^2} \cos(\omega_i x) \cos(\omega_i y), \end{aligned} \tag{6}$$

where $a_{i,j}$ is the density coefficient. $D_b(\mathbf{x}, \mathbf{y})$ is defined similarly. Since components have arbitrary shapes, a module drawing algorithm based on Bersenham's line drawing algorithm is used in the calculation [10]. However, the previous module drawing algorithm is not accurate. Further, TH pads must be considered on both sides due to their penetration. Therefore, we introduce our two-sided density model in Section IV.

III. Overall Flow

This paper presents a novel placement algorithm for PCB designs that is driven by clearance constraints and aware of heterogeneous components. As illustrated in Figure 3, our placement framework begins with input data comprising a netlist (i.e., schematic), a board library, a component library, and a pad-to-pad clearance table. The global placement algorithm spreads the components across the board to find a desired layout with slight overlaps and clearance violations. There are four stages in the global placement algorithm: obstacle insertion, initial placement, objective function evaluation, and gradient optimization. Finally, a legalization algorithm is employed to resolve any remaining constraint violations, ensuring a valid placement result. Detailed descriptions of these algorithms are provided in the following sections.

IV. Global Placement

In global placement, the process begins with obstacle insertion to convert a non-rectangular board into a rectangular placement area. Next, an initial placement is performed to find a configuration that minimizes the total wirelength. Following this stage, the objective function is iteratively evaluated, and gradient descent is applied to distribute the components. The iterative process continues until the components are sufficiently spread across the placement area. Each stage in our global placement algorithm is detailed in the following parts.

A. Obstacle Insertion

Since PCBs often have irregular shapes, converting a board into a standardized rectangular placement area is essential for efficient calculation. For a non-rectangular board B, represented by a set of counter-clockwise vertices, the rectangular placement area is determined as follows: (1) calculate the axis-aligned bounding box R of board B, (2) perform a geometry XOR operation between the polygons R and B to define the obstacles O, and (3) decompose O into a set of convex obstacles O'

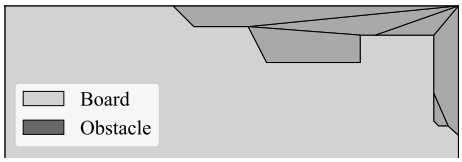


Fig. 4. An example of obstacle insertion for non-rectangular boards. Convex obstacles are added to create a rectangular placement area, preventing blocks from being placed outside the board.

using a convex decomposition algorithm. These obstacles contribute their areas to the density map $\rho(x, y)$ to prevent components from being placed in these restricted zones. Figure 4 illustrates an example of a board, along with the resulting placement area and obstacles generated by our algorithm.

B. Initial Placement

Following obstacle insertion, we seek an initial solution that minimizes the total XHPWL to establish a starting point for the main optimization process. All movable components are initially placed randomly near the center of the board. Gradient descent is then applied to iteratively minimize Equation 3, adjusting component positions to reduce the total wirelength.

C. Objective Function Evaluation

After initial placement, we aim to solve Equation 2 accurately with gradient descent algorithms. Therefore, a robust and accurate model for routing resources and density map is necessary. Moreover, a smooth and differentiable approximation model of the pad-to-pad clearance violation defined in Equation 4 is needed because the minimum distance calculation using GJK-EPA is not differentiable. These models are introduced in the following sections.

1) X Half-perimeter Wire-area Model: The XHPWL model does not accurately represent a net's consumed routing resource on a PCB, as the preferred routing wire width varies with nets. For instance, a power/ground net requiring a wider wire width consumes more routing resources than a signal net with the same wirelength. Consequently, components connected to nets with wider wire widths should ideally be positioned closer together. To address this problem, we propose an X half-perimeter wire-area (XHPWA) model $\tilde{W}(\mathbf{x}, \mathbf{y})$ in place of the XHPWL $W(\mathbf{x}, \mathbf{y})$, defined as follows:

$$\tilde{W}(\mathbf{x}, \mathbf{y}) = \sum_{e \in E} w_e l_e(\mathbf{x}, \mathbf{y}), \tag{7}$$

where w_e denotes the wire width of net e. This XHPWA model offers a more precise representation of routing resource consumption by considering varying wire width requirements across different nets, enabling better placement optimization.

2) Clearance Model: To achieve a smooth approximation for pad-to-pad clearance penalty $p(p_m, p_n)$, we use a bounding circle model to formulate a proximate penalty function $\tilde{p}(p_m, p_n)$. For each pad p_m, a bounding circle with radius r_m is pre-calculated. The proximate pad-to-pad clearance penalty function $\tilde{p}(p_m, p_n)$ can then be expressed as follows:

$$\begin{aligned} &\tilde{p}(p_m, p_n) = \\ &\quad \max\left(r_m + r_n + c_{m,n} - \sqrt{(x_m - x_n)^2 + (y_m - y_n)^2}, 0\right). \end{aligned} \tag{8}$$

The gradient of $\tilde{p}(p_m, p_n)$ with respect to x_m is given by:

$$\frac{\partial \tilde{p}(p_m, p_n)}{\partial x_m} = \begin{cases} -\frac{x_m - x_n}{\sqrt{(x_m - x_n)^2 + (y_m - y_n)^2}}, & \tilde{p}(p_m, p_n) > 0 \\ 0, & \tilde{p}(p_m, p_n) \leq 0 \end{cases}. \tag{9}$$

The gradients with respect to x_n, y_m, and y_n are derived similarly. Figure 5 illustrates our pad-to-pad clearance model applied in global placement. Importantly, any overhead introduced by the bounding circle model is mitigated in the legalization algorithm discussed in Section V. Additionally, to avoid undefined behavior, pad-to-pad clearance violations are excluded in two special cases: (1) when two pads belong to the same component, where there is no room for optimization, and (2) when two pads belong to the same net, as they will ultimately be connected.

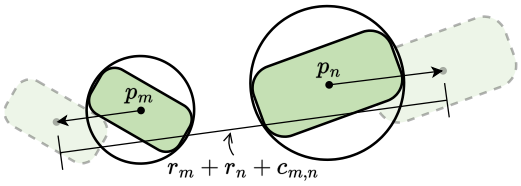


Fig. 5. Our pad-to-pad clearance model between two pads. Each pad, p_m and p_n, is represented by a bounding circle with radius r_m and r_n, respectively. The model ensures that p_m and p_n are moved apart if their center-to-center distance is less than the required spacing $r_m + r_n + c_{m,n}$.

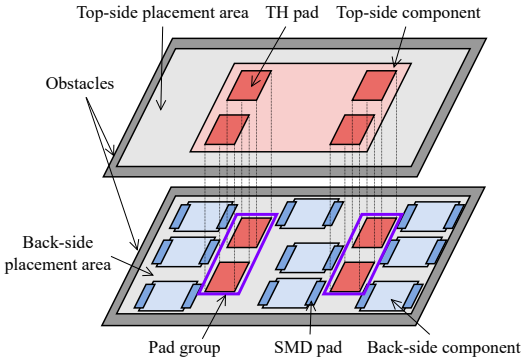


Fig. 6. Illustration of the two-sided density model. The model consists of two parts: the density penalty for the top side and the density penalty for the backside. Each density penalty corresponds to a placement area that includes obstacles, components on the same layer, and TH pads from the opposite side. TH pads are clustered into groups to prevent components from getting trapped between two TH pads.

3) Two-sided Density Model: We propose a two-sided density model to accurately distribute components on both sides of the board. Unlike previous models, ours accounts for interference from the opposite side due to TH pads and accurately calculates the density map using a coarsened grid. Figure 6 provides an overview of this two-sided density model. The back-side density penalty D_b spreads the blocks located in the back-side placement area, including the back-side components, SMD pads, and TH pads from the top side. The TH pads belonging to the same component will be further clustered into pad groups, effectively preventing back-side components from getting stuck in the middle of two TH pads. The top-side density penalty D_t is similarly defined.

Given that any shape, whether a component, a pad, or an obstacle, can be placed in various orientations and may need to be flipped (e.g., when placed on the back side), each shape is represented as a set of counter-clockwise vertices in a matrix form, M_s, as follows:

$$M_s = \begin{bmatrix} x \\ y \end{bmatrix} \mathbf{1} + M_f M_r M_o, \tag{10}$$

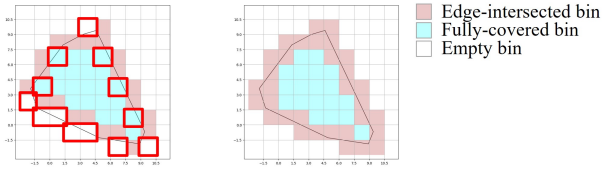
where (x, y) is the reference point (typically the component's center), and M_o, M_f, and M_r represent the contour offset matrix, flip matrix, and rotation matrix, respectively.

For non-convex shapes, we determine the need for conversion to a convex hull by calculating the area ratio $t = A_{real}/A_{cvx}$, where A_{real} is the actual area of the shape, and A_{cvx} is the area of its convex hull. If the ratio falls below a user-defined threshold, we partition the shape into convex sub-polygons; otherwise, we approximate the non-convex shape with its convex hull.

Once the exact contour of each shape is established, the density map $\rho(x, y)$ is updated using a supercover line-drawing algorithm from computational geometry [19]. The supercover line-drawing algorithm calculates the intersected bins by iterating through the adjacent vertices in M_s. This approach differs from previous methods by ensuring that every intersected bin is covered, preventing omissions. Figure 7 illustrates the difference between the prior approach [10] and our method. By covering all edge-intersected bins, our model enables efficient and robust density calculation with a coarsened grid.

D. Gradient Optimization

We employ a combination of Nesterov's method [18] and the Conjugate Gradient method [20] to optimize Equation 2. Nesterov's method, while achieving faster convergence, can become unstable as components spread gradually. In contrast, the Conjugate Gradient method converges more slowly but provides greater stability as optimization nears a steady point. Thus, Nesterov's method is applied in the initial phase of optimization, followed by the Conjugate Gradient method in the later phase.



(a) Previous work. (b) Ours.

Fig. 7. An illustration of the differences between the density calculation methods in previous work and our approach. (a) Density calculation in the previous work [10]. Mismatched bins, circled in red, lead to over- or under-estimation of density, affecting the spreading direction when using a coarse grid. (b) Our supercover density calculation algorithm accurately computes intersected bins, ensuring greater precision.

The initial values for the Lagrange multipliers λ_1 and λ_2 in Equation 2 are set as $|\nabla\tilde{W}|/|\nabla C_{p2p}|$ and $|\nabla\tilde{W}|/|\nabla D|$, respectively, to balance the gradient norms between the total wire area, the total pad-to-pad clearance violation, and the density penalty. λ_1 and λ_2 are incrementally increased by a user-defined factor (e.g., 1.007) whenever the objective function fails to improve every k times, with k typically set to 3.

The optimization progress is tracked using the overflow ratio, which measures the extent of overlapping areas in the current solution and is defined as follows:

$$\frac{\sum_{b\in\rho_{\text{top}}} \max(N_b - A_b, 0) + \sum_{b\in\rho_{\text{back}}} \max(N_b - A_b, 0)}{A_V + 2A_{\text{TH}}}, \tag{11}$$

where A_V and A_{TH} denote the total area of all components and all TH pads, respectively; ρ_{top} and ρ_{back} are the density maps for the top and back sides of the board; N_b is the area of components in bin b; and A_b represents the bin area. If the overflow ratio exceeds a user-defined threshold, Nesterov's method is applied. Otherwise, the Conjugate Gradient method is used. The global placement process is completed once the overflow ratio reaches or falls below a target threshold.

V. Legalization

After the global placement, we address remaining overlaps and pad-to-pad clearance violations using quadratic programming. Traditional legalization algorithms are often incompatible with heterogeneous components and pads, so we develop a quadratic program formulation utilizing the separating axis theorem to address these issues.

Given two convex shapes (i.e., components and pads), represented in matrix form as M_s^1 and M_s^2, according to the separating axis theorem, if two convex shapes do not intersect, there exists a line (the separating axis) defined by $\mathbf{u} \neq 0$, where $\mathbf{u} \in \mathbb{R}^2$ and $\gamma \in \mathbb{R}$, such that:

$$\mathbf{u}M_s^1 + \gamma \leq 0 \quad \text{and} \quad \mathbf{u}M_s^2 + \gamma \geq 0. \tag{12}$$

The position of each shape is expressed relative to its reference location using a linear equation (Equation 10). For non-intersecting shapes, we identify the separating axis using the GJK algorithm. For intersecting shapes, the separating axis is derived from the minimum penetration vector obtained through the EPA algorithm. Non-convex shapes are decomposed into convex sub-polygons, enabling us to find separating axes between each sub-polygon and other shapes. This approach effectively transforms non-overlapping constraints into linear constraints. Similarly, by adding a separating axis between a boundary obstacle O' and a shape, we ensure that the shape remains within the board area.

The pad-to-pad clearance constraints are represented as the minimum point-to-line distance between each shape and its corresponding separating axes. According to the point-to-line distance formula, if two shapes must be separated by a minimum distance $c_{m,n}$, the clearance constraints can be rewritten as follows:

$$\frac{\mathbf{u}M_s^1 + \gamma}{|\mathbf{u}|} \leq -\frac{1}{2}c_{m,n} \quad \text{and} \quad \frac{\mathbf{u}M_s^2 + \gamma}{|\mathbf{u}|} \geq \frac{1}{2}c_{m,n}. \tag{13}$$

Let $(\hat{x}_i, \hat{y}_i)$ denote the coordinates of a component v_i after legalization. The primary goal of legalization is to satisfy these constraints while maintaining alignment with the global placement result. Therefore, we

TABLE II

THE STATISTICS OF THE BENCHMARK SUIT USED IN NS-PLACE. #FIXED DENOTES THE NUMBER OF FIXED COMPONENTS. #LAYERS DENOTES THE NUMBER OF ROUTING LAYERS.

Design	Width	Height	$\|V\|$	#Fixed	#Pads	$\|E\|$	#Layers
PCB1	20.5	13.9	8	1	40	15	2
PCB2	50.8	22.9	18	5	77	34	2
PCB3	55.0	28.0	34	4	138	38	2
PCB4	22.0	60.0	28	6	140	52	2
PCB7	50.8	22.9	46	2	207	69	2
PCB10	101.6	53.3	57	16	319	99	2
PCB12	58.0	59.5	58	4	233	35	4
PCB13	86.0	71.5	61	3	314	63	4

minimize the displacement from the global placement, represented as the total quadratic moving distance:

$$(x-\hat{x})^2+(y-\hat{y})^2. \tag{14}$$

To enhance robustness, each separating axis is allowed to slide along its normal direction. This flexibility is incorporated by introducing a support variable $\hat{\gamma}$ for each separating axis and adding its quadratic sliding distance $(\gamma-\hat{\gamma})^2$ to the objective function.

Combining these elements, let Γ represent the set of all separating axes. Our quadratic programming model is formulated as follows:

$$\min_{\hat{\mathbf{x}},\hat{\mathbf{y}},\hat{\gamma}} \quad \sum_{v_i\in V}\left[(x_i-\hat{x}_i)^2+(y_i-\hat{y}_i)^2\right]+\sum_{(i,j)\in\Gamma}(\gamma_{i,j}-\hat{\gamma}_{i,j})^2, \tag{15}$$

$$\text{s.t.} \quad \mathbf{u}_{i,j}M_s^i+\hat{\gamma}_{i,j}\le 0 \quad \text{and} \quad \mathbf{u}_{i,j}M_s^j+\hat{\gamma}_{i,j}\ge 0, \quad \forall i\in V,\forall j\in O', \tag{16}$$

$$\mathbf{u}_{i,j}M_s^i+\hat{\gamma}_{i,j}\le 0 \quad \text{and} \quad \mathbf{u}_{i,j}M_s^j+\hat{\gamma}_{i,j}\ge 0, \quad \forall i,j\in V_t\cup P^T, \tag{17}$$

$$\mathbf{u}_{i,j}M_s^i+\hat{\gamma}_{i,j}\le 0 \quad \text{and} \quad \mathbf{u}_{i,j}M_s^j+\hat{\gamma}_{i,j}\ge 0, \quad \forall i,j\in V_b\cup P^T, \tag{18}$$

$$\frac{\mathbf{u}_{i,j}M_s^i+\hat{\gamma}_{i,j}}{|\mathbf{u}_{i,j}|}\le -\frac{c_{m,n}}{2} \quad \text{and} \quad \frac{\mathbf{u}_{i,j}M_s^j+\hat{\gamma}_{i,j}}{|\mathbf{u}_{i,j}|}\ge \frac{c_{m,n}}{2}, \quad \forall i,j\in P_t^S\cup P^T, \tag{19}$$

$$\frac{\mathbf{u}_{i,j}M_s^i+\hat{\gamma}_{i,j}}{|\mathbf{u}_{i,j}|}\le -\frac{c_{m,n}}{2} \quad \text{and} \quad \frac{\mathbf{u}_{i,j}M_s^j+\hat{\gamma}_{i,j}}{|\mathbf{u}_{i,j}|}\ge \frac{c_{m,n}}{2}, \quad \forall i,j\in P_b^S\cup P^T. \tag{20}$$

In this model, Equation 15 defines the objective function, while Equations 16, 17, 18, 19, and 20 specify the placement region constraints, top-side non-overlapping constraints, back-side non-overlapping constraints, and both top-side and back-side pad-to-pad clearance constraints, respectively.

The direction of each separating axis plays a crucial role in ensuring the feasibility of the quadratic programming model. To enhance the robustness of our legalizer, a two-step approach is adopted. In the first step, Equations 15, 16, 17, and 18 are solved to produce an initial non-overlapping layout. In the second step, the separating axes are updated based on the results of the first step, and Equations 15 to 20 are resolved using the updated axes to refine the placement. Further, TH pads are clustered into groups to prevent components from being trapped between two TH pads on opposite sides, further improving the robustness of the model. If any step results in an infeasible model, the output from the previous step is returned as the final solution.

VI. EXPERIMENTAL RESULTS

The proposed algorithm was implemented in C++ and Python, utilizing the OpenMP library for multi-threading. The Gurobi Optimizer [21] was employed as the quadratic programming solver. All experiments were conducted on a Linux workstation equipped with AMD Ryzen 3990X 2.9GHz processors and 128 GB of memory, with the number of CPU threads restricted to eight.

Our PCB placement algorithm was compared with two state-of-the-art academic placers: NS-Place [9] and the placer developed by Tsou *et al.* [11] Table II provides statistics for the benchmark suite used by

TABLE III

THE STATISTICS OF THE INDUSTRIAL BENCHMARK SUIT. #FIXED DENOTES THE NUMBER OF FIXED COMPONENTS, #LAYERS DENOTES THE NUMBER OF ROUTING LAYERS, AND P^S DENOTES THE SET OF TOP- AND BACK-SIDE SMD PADS.

Design	Width	Height	$\|V_t\|$	$\|V_b\|$	#Fixed	$\|P^S\|$	$\|P^T\|$	$\|E\|$	#Layers
CB1	48.0	48.0	14	44	5	98	33	36	1
CB2	82.8	35.2	15	72	4	175	31	53	1
CB3	102.0	44.5	20	69	5	157	42	50	1
CB4	124.5	22.0	108	98	5	520	27	135	2
CB5	129.0	79.1	39	159	12	357	99	108	2
CB6	68.2	30.5	88	119	3	533	55	128	4

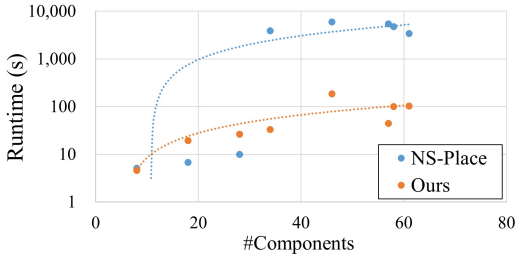


Fig. 8. The scalability comparison with NS-Place.

NS-Place and its predecessor, PCBRouter [22]. This benchmark suite was sourced from GitHub [23]. Due to confidentiality restrictions, PCB6, PCB11, and PCB14 were unavailable, as clarified by the original authors. Moreover, PCB5, PCB8, and PCB9 were removed because they were inconsistent with NS-Place. Note that we found some mismatches in the number of locked components compared to the values reported in NS-Place and made every effort to reconcile these differences.

Table III summarizes another benchmark suite that includes challenging modern industrial cases. These cases feature highly heterogeneous components across various design styles and demand strict pad-to-pad clearance constraints, making them suitable for comparison with the placer by Tsou *et al.*. **Note that achieving high routability in these industrial cases is hard, according to the provided company.** To evaluate the placement quality, the benchmarks in Table II were routed using FreeRouting [24], the same tool employed by NS-Place, and the design rule violations (DRVs) were evaluated using KiCad 6.0 [3]. The DRVs are listed as follows:

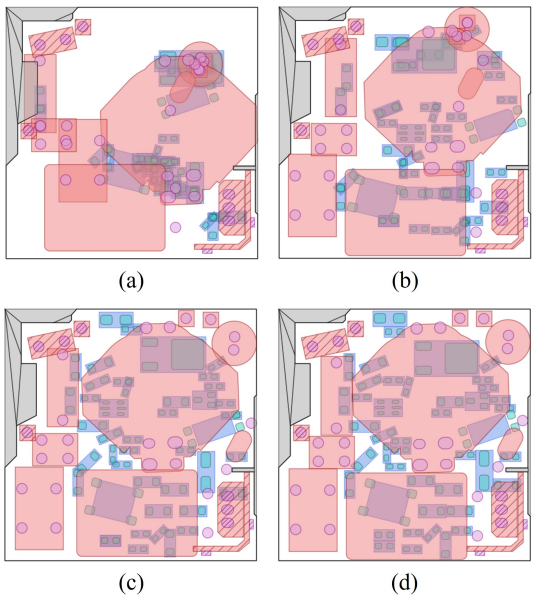


Fig. 9. The placement snapshots of CB1. (a) Initial placement: Components are centered and arranged to minimize the total XHPWL. (b) During global placement: Components are distributed by solving Equation 2. (c) After global placement: Components are positioned with minimal overlap and clearance constraint violations. (d) After legalization: Remaining overlaps and clearance violations are resolved.

TABLE IV
COMPARISON BETWEEN NS-PLACE AND OUR PLACER. HPWL, RT, PRWL, AND #DRVS DENOTE THE HPWL AFTER PLACEMENT, PLACEMENT RUNTIME, POST-ROUTING WIRELENGTH, AND NUMBER OF DESIGN RULE VIOLATIONS (INCLUDING UNROUTED/SHORTED NETS).

Design	NS-Place [9]					Ours				
	HPWL(mm)	RT(s)	PRWL(mm)	#Vias	#DRVs	HPWL(mm)	RT(s)	PRWL(mm)	#Vias	#DRVs
PCB1	72.10	5.10	121.00	4	0	**68.56**	4.61	**83.02**	1	0
PCB2	296.18	6.80	421.00	5	23	**275.88**	19.45	**296.57**	0	0
PCB3	**271.30**	3917.60	616.00	15	0	322.38	33.26	**463.84**	9	0
PCB4	**531.19**	9.90	806.00	54	0	667.50	26.55	**702.27**	9	0
PCB7	1601.40	6008.20	2949.00	93	0	**779.26**	184.73	**971.96**	51	0
PCB10	3315.46	5417.20	4914.00	97	0	**3228.83**	44.32	**3442.34**	73	0
PCB12	941.30	4752.30	1720.00	45	0	**885.74**	100.87	**1280.82**	15	0
PCB13	2151.77	3416.30	2897.00	83	0	**1876.17**	103.62	**2337.04**	61	0
Compare	1.00	1.00	1.00	1.00	-	**0.95**	**0.82**	**0.70**	**0.48**	-

TABLE V
COMPARISON BETWEEN TSOU ET AL.'S PLACER AND OUR PLACER. #CV DENOTES THE NUMBER OF CLEARANCE VIOLATIONS AND ROUT. DENOTES THE ROUTABILITY.

Design	Tsou *et al.* [11]					Ours				
	HPWL (mm)	XHPWA (mm^2)	#CV	RT (s)	Rout.	HPWL (mm)	XHPWA (mm^2)	#CV	RT (s)	Rout.
CB1	580.47	456.91	79	36.55	0.60	584.07	483.95	77	84.71	**0.82**
CB2	1176.25	1430.22	540	52.51	0.44	855.42	724.72	67	201.59	**0.81**
CB3	1330.15	1407.12	439	63.32	0.54	856.38	822.91	45	149.24	**0.82**
CB4	4661.46	1636.76	1557	84.91	0.40	1708.20	582.31	56	436.33	**0.90**
CB5	3185.41	3008.72	175	181.05	0.64	2085.04	1914.83	53	1005.18	**0.84**
CB6	4112.86	1365.94	1228	67.71	0.48	2031.99	682.82	29	409.92	**0.82**
Avg.	2507.77	1550.95	669.67	81.01	0.52	1353.52	868.59	54.50	381.16	**0.84**
Comp.	1.00	1.00	1.00	**1.00**	-	**0.65**	**0.61**	**0.26**	4.21	**-**

out-of-boundary blocks, overlapping components, pad-to-pad clearance violations, and open/short nets. The benchmarks in Table III were routed using the in-house commercial router applied in Tsou *et al.*'s work and evaluated the pad-to-pad clearance violations (CVs) using the same in-house design rule checker used by Tsou *et al.*.

A. Performance on Academic Benchmarks

Table IV compares the placement and routing results of our algorithm against NS-Place, with NS-Place results cited from the original paper [9]. Key metrics such as HPWL and runtime (RT) are used to evaluate placement effectiveness and efficiency, along with the post-routing wirelength (PRWL), the number of vias (#Vias), and the number of design rule violations (#DRVs) to assess the routability of the placement results. The results show that, on average (normalized), our placer achieved improvements of 5% in HPWL, 18% in runtime, 30% in PRWL, and 52% in via usage. Further, no DRVs were found in our results, whereas NS-Place reported 23 DRVs in PCB2.

The improvements in placement quality can be attributed to several factors: (1) NS-Place aims to separate each net's routing region, which often leads to over-spreading of components. (2) Our wire-area model accounts for the 135-degree routing nature, providing a more accurate estimation of routing resources. Figure 8 compares scalability, showing that our placer achieves a speedup of 33X to 100X in large cases. This performance gain is primarily due to the effectiveness of the quadratic programming legalizer we adopted, which outperforms the mixed-integer linear programming legalizer used in NS-Place.

B. Performance on Modern Industrial Benchmarks

Table V compares our algorithm with Tsou *et al.*'s placer, where the results for Tsou *et al.*'s placer were obtained by running their binary. Key metrics such as HPWL, XHPWA, the number of clearance violations (#CV), and runtime (RT) are used to evaluate placement effectiveness and efficiency. Routability is included to assess placement quality. The results show that, on average (normalized), our algorithm outperformed Tsou *et al.*'s placer by 35% in HPWL, 39% in XHPWA, 74% in clearance constraint violations, and 32% in routability. These improvements are attributed to the proposed models in Section IV, which handle clearance constraints and heterogeneous components more effectively. Unlike Tsou *et al.*'s approach, which addresses clearance constraints only during the legalization stage, our algorithm incorporates clearance constraints during global placement. This distinction reduces large displacements and improves routability, demonstrating the importance of addressing clearance constraints early in the placement process. The runtime overhead in our results is mainly due to constructing quadratic programs with separating axes. Our algorithm is shown to be a highly effective and efficient solution for capturing complex geometric relationships, as evidenced by the substantial reduction in clearance violations. Importantly, manually addressing these violations is both highly time-consuming and labor-intensive, demonstrating that the trade-off in runtime is necessary for achieving significant gains in modern PCB designs.

Figure 9 shows snapshots of the placement process with our algorithm. In the initial placement phase, blocks were centered on the board. During global placement, blocks were distributed by clearance and density constraints. Finally, the proposed quadratic programming legalizer resolved remaining overlaps and clearance violations, ensuring a high-quality placement result.

VII. CONCLUSION

This paper has presented a novel PCB placement framework considering pad-to-pad clearance constraints and heterogeneous components. Unlike existing academic placers, our framework has incorporated the following features: (1) a wire-area model to differentiate routing resource consumption between power and signal nets, (2) a pad-to-pad clearance model to minimize spacing violations, and (3) a two-sided, pad-type-aware density model to reduce component(pad) overlap. We have also developed a quadratic programming-based legalizer to remove constraint violations between components with arbitrary shapes. Experimental results have demonstrated the effectiveness and efficiency of our approach, with our algorithm outperforming two state-of-the-art academic placers in post-routing quality on both academic and industrial benchmarks. Specifically, our algorithm has shown an average improvement of 30% in post-routing wirelength over NS-Place, an average reduction of 74% in clearance violations over Tsou *et al.*'s work, and an average improvement of 32% in routability over Tsou *et al.*'s work. These results have justified the practical applicability of our PCB placement solution.

VIII. ACKNOWLEDGMENT

We thank Mr. Pin An Chen, Mr. Wei No Chen, Mr. Sean Huang, Ms. Ann Wei, Mr. Charles Liu, and Mr. Denver Liu of Delta Electronics for their great help in this research.

REFERENCES

[1] Allegro X design platform. [Online]. Available: https://www.cadence.com/en_US/home/tools/pcb-design-and-analysis/allegro-x-design-platform.html

[2] Altium designer. [Online]. Available: https://www.altium.com

[3] KiCad EDA: A cross-platform and open source electronics design automation suite. [Online]. Available: https://www.kicad.org

[4] W.-H. Liu, A. Agnesina, and H. M. Ren, "Challenges for automating PCB layout," in *Proc. of ISPD*, Taipei, Taiwan, March 2024, pp. 91–92.

[5] Y.-W. Chang, "Physical design challenges in modern heterogeneous integration," in *Proc. of ISPD*, Taipei, Taiwan, March 2024, pp. 125–134.

[6] Renesas. [Online]. Available: https://www.renesas.com/en

[7] F. S. Ismail, R. Yusof, and M. Khalid, "Optimization of electronics component placement design on PCB using self organizing genetic algorithm (SOGA)," *Springer JOIM*, vol. 23, no. 1, pp. 883–895, 2012.

[8] T. Badriyah, F. Setyorini, and N. Yuliawan, "The implementation of genetic algorithm and routing Lee for PCB design optimization," in *Proc. of ICIC*, Mataram, Indonesia, October 2016, pp. 148–153.

[9] C.-K. Cheng, C.-T. Ho, and C. Holtz, "Net separation-oriented printed circuit board placement via margin maximization," in *Proc. of ASPDAC*, Taipei, Taiwan, January 2022, pp. 288–293.

[10] F. Huang, D. Liu, X. Li, B. Yu, and W. Zhu, "Handling orientation and aspect ratio of modules in electrostatics-based large scale fixed-outline floorplanning," in *Proc. of ICCAD*, San Francisco, California, October/November 2023, pp. 1–9.

[11] C.-H. Tsou, S.-Y. Lin, W.-C. Hung, and Y.-W. Chang, "Late breaking results: Modern automatic PCB placement with complex constraints," in *Proc. of DAC*, San Francisco, California, June 2024, pp. 1–2.

[12] Y.-J. Chen, C.-H. Hsieh, P.-H. Su, C. Shao-Hsiang, and Y.-W. Chang, "Mixed-size 3D analytical placement with heterogeneous technology nodes," in *Proc. of DAC*, San Francisco, California, June 2024, pp. 1–6.

[13] M. Su, Y. Xiao, S. Zhang, H. Su, J. Xu, H. He, Z. Zhu, J. Chen, and Y.-W. Chang, "Subgraph matching based reference placement for PCB designs: late breaking results," in *Proc. of DAC*, San Francisco, California, July 2022, pp. 1–2.

[14] T.-C. Chen, Y.-L. Chuang, and Y.-W. Chang, "X-architecture placement based on effective wire models," in *Proc. of ISPD*, Austin, Texas, March 2007, pp. 87–94.

[15] M.-K. Hsu, V. Balabanov, and Y.-W. Chang, "TSV-aware analytical placement for 3-D IC designs based on a novel weighted-average wirelength model," *IEEE TCAD*, vol. 32, no. 4, pp. 497–509, 2013.

[16] E. G. Gilbert, D. W. Johnson, and S. S. Keerthi, "A fast procedure for computing the distance between complex objects in three-dimensional space," *IEEE JORAA*, vol. 4, no. 2, pp. 193–203, 1988.

[17] G. Van Den Bergen, "Proximity queries and penetration depth computation on 3D game objects," in *Proc. of GDC*, San Jose, California, March 2001, p. 209.

[18] J. Lu, P. Chen, C.-C. Chang, L. Sha, D. J.-H. Huang, C.-C. Teng, and C.-K. Cheng, "ePlace: Electrostatics-based placement using fast fourier transform and Nesterov's method," *ACM TODAES*, vol. 20, no. 2, pp. 1–34, 2015.

[19] Bresenham-based supercover line algorithm. [Online]. Available: http://eugen.dedu.free.fr/projects/bresenham

[20] T.-C. Chen, Z.-W. Jiang, T.-C. Hsu, H.-C. Chen, and Y.-W. Chang, "NTUplace3: An analytical placer for large-scale mixed-size designs with preplaced blocks and density constraints," *IEEE TCAD*, vol. 27, no. 7, pp. 1228–1240, 2008.

[21] Gurobi optimizer reference manual. [Online]. Available: http://www.gurobi.com

[22] T.-C. Lin, D. Merrill, Y.-Y. Wu, C. Holtz, and C.-K. Cheng, "A unified printed circuit board routing algorithm with complicated constraints and differential pairs," in *Proc. of ASPDAC*, Tokyo, Japan, January 2021, pp. 170–175.

[23] PCB benchmarks. [Online]. Available: https://github.com/aspdac-submission-pcb-layout/PCBBenchmarks

[24] FreeRouting. [Online]. Available: https://freerouting.org